LINUX COMMAND LINE · PRACTICE SET

Linux CLI Practice Problems

Ten problems for hands-on practice. Each one gives you a file to create and a list of tasks to solve at the terminal. No answers are included: work them out yourself.

10	51	bash
PROBLEMS	TASKS	ANY LINUX TERMINAL

BEFORE YOU START

1. Make a practice folder: `mkdir -p ~/linux_practice && cd ~/linux_practice`
2. Type or paste each **Create file** block exactly as shown. The last `EOF` must be alone on its line.
3. Solve every task with **one command or one pipeline**. Don't open the files in an editor.
4. Do the tasks in order. In a few problems, one task changes the file used by the next.
5. Some files hide small traps. If a result looks odd, check the data before blaming the command.

#	PROBLEM	COMMANDS YOU'LL NEED	LEVEL
01	**grep: beyond the basics**	`grep` `cut`	EASY
02	**head and tail: slicing a file**	`head` `tail`	EASY
03	**Redirection and tee**	`>` `>>` `2>` `tee`	EASY
04	**tr and wc: changing characters**	`tr` `wc`	EASY
05	**CSV data: cut, sort and uniq**	`tail` `cut` `sort` `uniq` `awk`	MEDIUM
06	**sed: cleaning up a config file**	`sed` `grep` `wc`	MEDIUM
07	**awk: totals per group**	`awk` `sort` `head`	MEDIUM
08	**find: acting on what you find**	`find` `wc`	MEDIUM
09	**Comparing files: diff and comm**	`diff` `comm` `sort` `wc`	HARDER
10	**Final challenge: regression report**	any of the above	HARDER

PROBLEM 01 EASY

grep: beyond the basics

Commands you'll need: `grep` `cut`

CREATE FILE

```
cat > tests.txt << EOF
test_reset PASS
test_burst FAIL
test_reset_async PASS
test_write PASS
test_read FAIL
test_burst_wrap PASS
test_write_read FAIL
EOF
```

TASKS

1. Show every line that does **not** contain PASS.
2. Show only the line for `test_reset`, not `test_reset_async`.
3. Show the tests whose name contains `burst` or `read`, using one grep.
4. Count the tests whose name **starts with** `test_write`.
5. Print only the names of the failed tests, without the word FAIL.

PROBLEM 02 EASY

head and tail: slicing a file

Commands you'll need: `head` `tail`

CREATE FILE

```
cat > months.txt << EOF
January
February
March
April
May
June
July
August
September
October
November
December
EOF
```

TASKS

1. Show the first 4 months.
2. Show the last 3 months.
3. Show only May to August, using **only** head and tail.
4. Show every month except January.
5. Show every month except the last two.

PROBLEM 03 EASY

Redirection and tee

Commands you'll need: `>` `>>` `2>` `tee`

`missing.txt` does not exist. That is on purpose.

CREATE FILE

```
cat > notes.txt << EOF
first line
second line
EOF
```

TASKS

1. Add the line "third line" to the end of notes.txt without losing what is already there.
2. Run `cat notes.txt missing.txt`. Save the normal output in out.txt and the error message in err.txt.
3. Run the same command again, but throw the error message away so only the normal output appears on screen.
4. Save **both** the normal output and the error of that command into one file, all.txt.
5. Show the line count of notes.txt on screen **and** save it in count.txt with the same command.

PROBLEM 04 EASY

tr and wc: changing characters

Commands you'll need: `tr` `wc`

CREATE FILE

```
cat > message.txt << EOF
linux is fun.....
practice every day
the terminal is your friend!!!
EOF
```

TASKS

1. Show the whole file in UPPERCASE.
2. Squeeze the repeated dots and exclamation marks down to one each.
3. Show the file with all vowels removed.
4. Print one word per line.
5. Count the words in the file, then the characters.

PROBLEM 05 MEDIUM

CSV data: cut, sort and uniq

Commands you'll need: `tail` `cut` `sort` `uniq` `awk`

The first line is a header, not a student.

CREATE FILE

```
cat > students.csv << EOF
name,dept,marks
Ravi,ECE,78
Priya,CSE,91
Arun,ECE,85
Divya,EEE,67
Karthik,CSE,91
Meena,ECE,59
Suresh,EEE,73
EOF
```

TASKS

1. Print the data without the header line.
2. Print only the name and marks columns.
3. Sort the students by marks, highest first, leaving out the header.
4. Count how many students are in each department.
5. Print the name of the topper. If there is a tie, print every topper.

PROBLEM 06 MEDIUM

sed: cleaning up a config file

Commands you'll need: `sed` `grep` `wc`

printf writes each quoted string on its own line, so each `""` makes a blank line.

CREATE FILE

```
printf '%s\n' \
  "# simulation settings" \
  "SEED=1" \
  "VERBOSITY=LOW" \
  "" \
  "TIMEOUT=1000" \
  "# coverage" \
  "COVERAGE=OFF" \
  "" \
  "# end of file" > sim.cfg
```

TASKS

1. Show the file without the comment lines (lines starting with #).
2. Show only the real settings: no comments and no blank lines, in one command.
3. Print only lines 2 to 5.
4. Change SEED to 42 and VERBOSITY to HIGH, saving both changes into the file with one command.
5. Count how many settings the file has.

PROBLEM 07 MEDIUM

awk: totals per group

Commands you'll need: `awk` `sort` `head`

Columns: order ID, city, amount.

CREATE FILE

```
cat > orders.txt << EOF
1001 Chennai 2500
1002 Bangalore 1800
1003 Chennai 3200
1004 Mumbai 900
1005 Bangalore 4100
1006 Chennai 1500
1007 Mumbai 2600
EOF
```

TASKS

1. Print all orders from Chennai.
2. Calculate the total amount of the Chennai orders only.
3. Count how many orders are above 2000.
4. Print the total amount for each city.
5. Find the city with the highest total.

PROBLEM 08 MEDIUM

find: acting on what you find

Commands you'll need: `find` `wc`

CREATE THE FILES

```
mkdir -p project/src project/docs project/build
echo "int main() { return 0; }" > project/src/main.c
echo "void helper() {}" > project/src/util.c
touch project/src/config.h
echo "Build notes" > project/docs/notes.txt
touch project/build/main.o project/build/util.o
```

TASKS

1. Find all .c files inside project.
2. Find all empty files.
3. List only the folders directly inside project (not project itself, nothing deeper).
4. Count the total number of lines in all the .c files together.
5. Delete every .o file with a single find command, then show that they are gone.

PROBLEM 09 HARDER

Comparing files: diff and comm

Commands you'll need: `diff` `comm` `sort` `wc`

Each file lists the tests that failed in one regression run.

CREATE THE FILES

```
cat > yesterday.txt << EOF
test_burst
test_read
test_reset
test_write
EOF
cat > today.txt << EOF
test_timeout
test_burst
test_write
test_read_wrap
test_reset
EOF
```

TASKS

1. Check whether the two files are identical, printing only a one-line answer.
2. Show the tests that failed on **both** days.
3. Show the **new** failures: failed today but not yesterday.
4. Show the **fixed** tests: failed yesterday but not today.
5. Count the new failures.

PROBLEM 10 HARDER

Final challenge: regression report

Commands you'll need: any command from the earlier problems

Each line: test name, seed, result, runtime in seconds.

CREATE FILE

```
cat > regression.log << EOF
test_reset seed=101 PASS 12
test_burst seed=102 FAIL 45
test_write seed=103 PASS 30
test_read seed=104 PASS 28
test_burst seed=105 FAIL 47
test_reset_async seed=106 PASS 15
test_timeout seed=107 FAIL 120
test_write seed=108 PASS 31
test_read seed=109 FAIL 29
test_burst_wrap seed=110 PASS 52
EOF
```

TASKS

1. Count the total runs, the passes and the failures.
2. Calculate the pass rate as a percentage.
3. List each failing test name once, with no duplicates.
4. Print the seeds of the failing runs as plain numbers (102, not seed=102).
5. Find the slowest run: its test name and runtime.
6. Calculate the average runtime.